

Adaptive, Intelligent Multi-Agent System for Autonomous Software Planning, Code Generation, and Self-Debugging

R26-SE-029

Project Proposal Report

Bavithran S – IT22115966

B.Sc. (Hons) Degree in Information Technology Specialized in
Software Engineering

Department of Computer Science & Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

DECLARATION

I hereby declare that this research proposal titled “**Adaptive Orchestration Engine and Episodic Memory for Autonomous Multi-Agent Software Generation**” is my own work and has not been submitted previously for any academic qualification at this or any other institution. The work presented in this proposal has been carried out by me under the guidance of my supervisor. All sources of information used in this research have been properly acknowledged and cited.

I confirm that this research proposal is submitted in partial fulfillment of the requirements for the relevant academic program.

Name	Student ID	Signature
Bavithran S	IT22115966	<i>Bavithran</i>

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Sanjeevi Chandrasiri	20.03.2026	<i>Sanjeevi</i>

ABSTRACT

Building backend software is currently a very manual, slow, and error-prone process. As projects grow larger, developers struggle to manage the workload. While modern AI tools can help write small pieces of code, a major research gap exists: they cannot manage the entire development workflow autonomously. When current AI systems encounter a bug during testing, they often get stuck in endless loops because they lack strong orchestration and memory.

To solve this problem, my research focuses on building an **Adaptive Agentic Orchestration Engine**. This engine acts as a smart manager that controls different AI agents—such as a planner, coder, and tester—while incorporating Human-in-the-Loop checkpoints for architectural validation. It is built using a Finite State Machine (FSM) via LangGraph to ensure the workflow stays on track without crashing. A key component of this architecture is the **Diagnostic Critic Agent**, which is dedicated solely to debugging and self-correction. When the testing agent finds a bug, the Orchestrator routes the raw error log to the Critic Agent. The Critic Agent then queries an **Episodic Memory system**—a runtime Vector Database of past "Error-Success" pairs and execution traces. By analyzing the current failure against this retrieved historical context, the Critic Agent synthesizes a highly detailed **Self-Correction Strategy**, allowing the coding agent to fix its mistake precisely and in real-time.

The system is developed using Python, LangGraph for workflow control, and a local Vector Database, specifically targeting the creation of Node.js REST APIs. The expected outcome is a reliable, self-healing software generation system that prevents deadlocks and "agentic mode collapse." Commercially, this product will provide massive value to software agencies by automating the manual debugging phase, reducing both technical debt and project costs. Socially, it will significantly reduce developer burnout by handling repetitive bug-fixing tasks.

Keywords: Multi-Agent System, Orchestration, Episodic Memory, Self-Correction, Critic Agent, Node.js, LangGraph

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my research supervisor, **Sanjeevi Chandrasiri** and our co-supervisor **Karthiga Rajendran**, for their invaluable guidance, continuous support, and encouragement throughout the preparation of this research proposal. Their expertise, insights, and constructive feedback have been instrumental in shaping the direction and quality of this study.

I would also like to extend my thanks to my friends and colleagues who are software engineering developers. Their practical advice, technical insights, and helpful discussions have greatly contributed to my understanding of software development concepts and the implementation aspects relevant to this research.

Furthermore, I am grateful to the academic staff of the faculty for providing the necessary resources, knowledge, and guidance required to complete this proposal successfully. Their lectures and advice have enriched my understanding of the research topic.

Finally, I wish to express my heartfelt appreciation to my family for their patience, encouragement, and moral support, which has been a source of motivation throughout this research journey.

TABLE OF CONTENT

Contents

DECLARATION	2
ABSTRACT	3
ACKNOWLEDGEMENT	4
TABLE OF CONTENT	5
LIST OF TABLES	6
LIST OF FIGURES.....	7
LIST OF ABBREVIATIONS.....	8
LIST OF APPENDICES	9
1.INTRODUCTION	10
2. OBJECTIVE	15
3. METHODOLOGY	16
4. HIGH-LEVEL SYSTEM ARCHITECTURE	19
5. USER REQUIREMENTS	21
6. COMMERCIALIZATION PLAN	22
7. BUDGET AND JUSTIFICATION	23
8. WORK BREAKDOWN STRUCTURE (WBS).....	25
9. GANTT CHART	27
10. REFERENCES LIST	28
11. APPENDICES.....	29

LIST OF TABLES

Table 1 - Tools & Platforms	17
Table 2 - Total Estimated Budget.....	24

LIST OF FIGURES

Figure 1 - System architecture Diagram	19
Figure 2- Individual Component.....	20
Figure 3- Work Breakdown Structure	26
Figure 4 - GANTT Chart	27

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
FSM	Finite State Machine
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SDLC	Software Development Life Cycle

LIST OF APPENDICES

Appendix	Description	Page
Appendix – A	Risk Assessment and Mitigation	29
Appendix – B	Ethical Compliance and Data Governance	29

1. INTRODUCTION

1.1 Background and Context

In the modern digital era, the traditional Software Development Life Cycle (SDLC) remains a highly manual, labor-intensive, and error-prone process. As software systems grow in complexity, building robust backend architectures—such as Node.js REST APIs—requires meticulous planning, extensive boilerplate coding, and rigorous testing. This manual approach makes development inherently slow and vulnerable to human error. Software Engineers, QA teams, and Technical Leads are frequently forced to coordinate complex tasks under tight deadlines. Consequently, developers spend approximately 50% of their programming time strictly on debugging rather than writing new code. For businesses, this translates to delayed deliveries, compromised reliability, and staggering financial impacts.

While the industry has tried to solve this with Artificial Intelligence, existing solutions have major limitations. AI Assistants like GitHub Copilot are passive, requiring constant human supervision. Early Multi-Agent Systems (like ChatDev) assign roles to different AI agents but suffer from "weak orchestration." They operate in rigid, linear pipelines. If a runtime error occurs, these systems cannot adapt. They frequently crash, get stuck in infinite loops, or suffer from "confirmation bias"—repeatedly trying the same broken solution due to a lack of long-term memory and a proper feedback loop.

To resolve these critical limitations, this research proposes an Adaptive, Intelligent Multi-Agent System incorporating Human-in-the-Loop checkpoints for architectural approval. My specific contribution focuses on three core components. First, I am developing the Adaptive Orchestration Engine, a central manager built using a LangGraph Finite State Machine. This engine controls the workflow between planning, coding, and testing agents using strict "retry budgets" to prevent infinite loops. Second, an Episodic Memory system utilizing a local Vector Database act as the active runtime memory, storing logs of past "Error-to-Success" pairs. Finally, a specialized Diagnostic Critic Agent handles advanced self-correction. When the testing sandbox finds a bug, the Orchestrator routes the raw error directly to the Critic Agent. This agent queries the Episodic Memory to analyze past resolutions and synthesizes a highly specific, context-aware repair strategy, allowing the system to intelligently debug itself in real-time.

This research strongly aligns with the United Nations Sustainable Development Goal 9 (Industry, Innovation, and Infrastructure) by fostering resilient technological infrastructure development. The project is conducted under the Software Systems & Technologies (SST) research cluster, pushing the boundaries of autonomous software engineering.

Keywords: Multi-Agent System, Adaptive Orchestration, Finite State Machine, Episodic Memory, Diagnostic Critic Agent, LangGraph, Node.js

1.2 Literature Review

The traditional software development lifecycle relies heavily on manual human effort for planning, implementation, and testing. However, recent advancements in large language models (LLMs) and artificial intelligence have facilitated a transition toward automated code generation. These AI-driven systems aim to streamline software engineering by reducing manual overhead, accelerating development timelines, and minimizing human error.

Automated software generation utilizes AI to autonomously execute tasks across the development pipeline, ranging from architectural design to testing and debugging. To manage the complexity of real-world software, these models are increasingly structured as multi-agent architectures, where specialized AI roles collaborate on distinct components of a project. The reliability and functional accuracy of such autonomous frameworks depend fundamentally on two core components: an intelligent orchestration engine to govern agent workflows and an episodic memory system to enable iterative self-correction. Consequently, an evaluation of current methodologies, architectural frameworks, and commercial products is necessary to identify the existing limitations in multi-agent software generation.

1.2.1 Existing Research Approaches

Several studies have explored the potential of using finite state machines (FSMs) to control multi-agent workflows. One such study [7] proposed a novel framework that conceptualizes complex task-solving as state machines, distinguishing between moving through structural steps and actually solving the sub-tasks. By separating the rules of progression from the coding itself, the system gains much tighter control over the development process. Different research [11] investigated the usage of an automated multi-agent system equipped with condition verifiers. In their model, if an agent produces incorrect output, the system triggers a "null-transition," forcing the agent to stay in its current state and fix the error locally before moving forward to the next phase. In addition, a study [8] proposed using declarative decoding monitors to formally specify high-level behaviors, ensuring agents do not accidentally skip mandatory phases like testing.

For an autonomous system to fix its own bugs over a long session, dynamic memory and self-correction are essential. A notable study [5] introduced a framework where language agents verbally reflect on task feedback, such as failed unit tests, and store these reflections in an episodic memory buffer to improve future decisions. Furthermore, research [10] demonstrated that integrating a lightweight strategic memory module preserves tactical coherence across different decision cycles. Another study [13] showed the effectiveness of teaching models to self-debug by generating execution traces and explaining their code line-by-line, functioning much like human "rubber duck debugging."

To ensure the full system operates smoothly without errors, researchers have also focused on role separation. A study [1] explored a communicative framework where specialized agents chat with each other to develop software, though unstructured communication can sometimes lead to coding hallucinations. To solve this, different research [2] required agents to follow Standardized Operating Procedures, producing structured documents like system architectures before writing code. Similarly, a study [3] proposed separating the coding and

testing phases by using an independent test designer agent, preventing the tests from inheriting the flawed logic of the generated code. Additionally, research [12] highlighted that single-agent debugging suffers from confirmation bias, proving that a multi-agent debate produces vastly superior diagnostic feedback. **This underscores the necessity of isolating the debugging logic into a specialized 'critic' role rather than relying on the coding agent to fix its own mistakes.** Other frameworks [4, 9] emphasize using execution results and multi-turn reviews to continuously repair semantic errors.

1.2.2 Existing Products and Services

While academic research focuses heavily on fully autonomous multi-agent orchestration, the commercial market for AI-assisted software development currently offers varying levels of autonomy. GitHub Copilot has evolved from a simple code-completion tool into a comprehensive developer suite. Recent updates introduced "Agentic Memory," allowing Copilot to build a persistent, repository-level understanding of a codebase so developers spend less time explaining context. However, it still functions primarily as an interactive assistant rather than a fully autonomous orchestration engine capable of managing a global project state.

Another prominent tool, Cursor AI, operates as a deeply integrated AI-native IDE. It utilizes advanced codebase indexing and features a multi-file editing mode that allows developers to execute broad changes simultaneously. While Cursor offers background agent modes for parallel tasks, it still relies on the developer to act as the primary orchestrator who guides the architectural boundaries. Similarly, Google Antigravity is a recently released agent-first platform that provides an "Agent Manager" view to spawn parallel workers across different workspaces. Despite this advanced parallel execution, it still positions the human developer as the main architect who must manually unblock agents when they encounter severe logic loops, lacking a fully closed-loop recursive engine.

1.2.3 Limitations of Existing Systems

Despite rapid advancements, current multi-agent architectures exhibit critical limitations in self-repair. A study [6] found self-correction frequently fails because single models struggle to introspect accurately. Deep repair loops remain ineffective unless a dedicated orchestrator provides synthesized feedback. Without a specialized critic mechanism diagnosing errors, systems inevitably succumb to agentic mode collapse, repeatedly attempting flawed solutions.

Furthermore, many systems operate on rigid, linear pipelines, falling into infinite reasoning loops during runtime failures due to a lack of FSM-driven state traceback mechanisms. Additionally, while commercial tools use static databases, they lack an active episodic vector store to index execution feedback. Consequently, previously resolved complex errors cannot be recalled if they reappear. These gaps strongly justify a novel adaptive orchestration engine integrating episodic memory and a Diagnostic Critic Agent to ensure reliable, autonomous software generation.

1.3 Research Gap

While significant progress has been made in utilizing large language models for automated code generation, a critical gap remains in achieving true autonomy across the entire software development lifecycle. Current research and commercial tools have successfully developed intelligent agents for isolated tasks, but they lack a robust, self-managing architecture that can coordinate these tasks without human intervention.

What is Missing

The primary missing component in current multi-agent software development is a dynamic, closed-loop orchestration engine paired with an active episodic memory system. Existing frameworks generally lack the ability to deterministically route agents through complex workflows, handle unexpected runtime errors gracefully, and remember successful debugging strategies across a long-running session.

Insufficiency of Current Approaches

Current approaches fall short in several key areas. First, widely used commercial tools like GitHub Copilot and Cursor AI function as highly advanced, interactive pair programmers. They excel at localized code completion and codebase retrieval but remain fundamentally tethered to a human developer to guide the architectural boundaries and resolve deep logical conflicts. They do not operate as fully autonomous software factories.

Second, within academic research, many multi-agent systems operate on rigid, linear pipelines. When these systems encounter an unexpected runtime failure, they frequently halt or fall into infinite, costly reasoning loops. They lack the strict state traceback mechanisms provided by a finite state machine (FSM). Furthermore, relying on a single agent to generate code, evaluate its own output, and apply fixes consistently leads to confirmation bias. Research highlights that self-correction is heavily bottlenecked by a single model's inability to accurately introspect, meaning self-repair often fails without a dedicated orchestrator to provide high-quality, synthesized feedback. Finally, while some systems use static databases to retrieve initial context, they fail to maintain a dynamic runtime memory. When an agent successfully resolves a complex dependency error, that knowledge is discarded; if the same error reappears later, the system must waste computational resources solving it from scratch.

Justification of Novelty and Creativity

This research addresses these exact limitations by proposing a novel Adaptive Orchestration Engine driven by a Finite State Machine (FSM), integrated with a persistent Episodic Memory buffer and Human-in-the-Loop verification nodes. Instead of relying on unstructured communication or linear prompts, the proposed FSM deterministically controls the transitions between the planning, coding, and testing phases. If a testing agent detects a failure, the orchestrator triggers a controlled "null-transition" or state traceback, forcing local refinement rather than allowing the system to derail. The creativity of this approach lies in the integration of episodic memory directly into the self-correction loop. As the multi-agent system encounters runtime errors and eventually synthesizes successful fixes, the orchestration engine logs these "Error-to-Fix" pairs into a runtime vector store. This creates a self-healing system that actively learns during a session.

Evidence-Based Argument

The necessity of this approach is strongly supported by the literature. Studies demonstrate that unstructured multi-agent communication leads to severe coding hallucinations [1], necessitating rigid structural control. Furthermore, research proves that self-repair is not a guaranteed solution; it requires a structured environment to prevent mode collapse and confirmation bias [6], [12]. By bridging the gap between FSM-driven state management [7], [11] and verbal reinforcement learning [5], this project moves beyond simple code-generation scripts. It provides a cohesive, autonomous framework capable of planning, generating, and dynamically self-debugging complex software architectures, significantly advancing the current boundaries of automated software engineering

2. OBJECTIVE

2.1 Main Objective

To design, develop, and evaluate an autonomous multi-agent software generation system that integrates context-aware planning, generative coding, sandboxed testing, and an adaptive orchestration engine incorporating Human-in-the-Loop validation to measurably reduce human intervention in the software development lifecycle.

2.2 Specific Objectives

- To integrate a RAG-based planning agent that transforms high-level requirements into structured technical blueprints through iterative human-agent collaborative sessions.
- To implement a multi-file synthesis agent that translates blueprints into functional, interconnected source code following industry-standard Node.js architectures.
- To incorporate a secure, containerized testing environment for autonomous code execution, runtime behavior monitoring, and precise error logging.
- To develop an FSM-driven engine using LangGraph to govern deterministic state transitions and manage the global project lifecycle with a robust feedback loop.
- To engineer a specialized **Diagnostic Critic Agent** coupled with a runtime **episodic memory buffer** (Vector Database) that enables the system to autonomously retrieve past error-to-success pairs, synthesize targeted repair strategies, and resolve execution errors without agentic mode collapse.

3. METHODOLOGY

3.1 Research Approach

This project follows a Design Science Research approach. This means we are not just observing a problem; we are building a new, functional "artifact" (the multi-agent system) to solve the gap in autonomous software development. The research focuses on creating a "brain" for the system that can reason, learn from its own mistakes, and manage a workflow without stopping when an error occurs. Unlike typical linear systems, this approach is experimental, as we will continuously test how well the self-correction loop handles different types of coding errors while incorporating Human-in-the-Loop validation nodes for architectural oversight.

3.2 Technical Methods and Algorithms

The Adaptive Orchestration Engine utilizes three primary technical methodologies to govern the autonomous multi-agent system:

- **Deterministic Workflow (LangGraph FSM):** A Finite State Machine controls the strict transition between Planning, Coding, and Testing. This prevents the system from getting stuck in infinite reasoning loops and enforces strict "retry budgets" for bug fixing.
- **Active Episodic Memory (Vector Database):** A runtime memory system that logs successful "Error-to-Fix" pairs. If the system encounters a similar bug later, it instantly retrieves the past solution rather than solving it from scratch, significantly speeding up development.
- **Diagnostic Critic Agent:** A specialized self-correction agent that intercepts raw test failures. Instead of blindly passing errors back to the coder, it queries the Episodic Memory, diagnoses the root cause, and writes a highly targeted "Self-Correction Strategy," escalating to a human only if the retry budget is exhausted.

3.3 Tools and Platforms

The orchestration engine uses three primary technical methods to govern the system:

Tool / Technology	Purpose
LangGraph	Manages the system's step-by-step workflow and decision-making logic.

Tool / Technology	Purpose
Python / FastAPI	The core programming language and framework used to build the central engine.
Vector DB (ChromaDB / FAISS)	Acts as the AI's memory to store and instantly recall past bug fixes.
Docker	A secure, isolated "sandbox" where the AI can safely test code without crashing the host computer.
LLM Selection	Currently testing models like GPT-4o, Claude 3.5, and DeepSeek to find the smartest and fastest AI.

Table 1- Tools & Platforms

3.4 Data Collection and Ethical Considerations

The project relies on two primary classifications of data to train and evaluate the multi-agent system:

- **Reference Data (Architectural Benchmarks):** We will curate high-quality, open-source Node.js and Express repositories from GitHub. These datasets will serve as "Gold Standard" architectural templates for the Planning Agent and its Retrieval-Augmented Generation (RAG) pipeline.
- **Execution Data (Episodic Memory Logs):** During runtime testing, the system will autonomously log raw execution errors alongside their corresponding successful fixes. This empirical "Experience Data" is indexed directly into the Vector Database, allowing the Diagnostic Critic Agent to dynamically improve its self-correction accuracy over time without the need for computationally expensive model fine-tuning.

Ethical Considerations Because the proposed architecture inherently relies on Human-in-the-Loop (HITL) checkpoints for architectural validation, formal institutional ethical clearance will be obtained prior to user testing. The evaluation phase will involve volunteer software engineering undergraduate students interacting with the orchestration engine. We will ensure that all participation is strictly voluntary, and any qualitative feedback or interaction metrics collected during these sessions will be thoroughly anonymized to protect participant privacy and maintain academic integrity.

3.5 Validation Strategy and Performance Metrics

prove the system works, we will measure it using the following criteria:

- **Success Rate of Self-Healing:** The percentage of runtime errors the system fixes autonomously without human help.
- **Reduction in Reasoning Loops:** Measuring if the Episodic Memory helps the system avoid repeating the same mistakes (reducing "mode collapse").
- **Retry Budget Efficiency:** How many attempts it takes the system to generate a fully functional, verified backend application.
- **Functionality vs. Syntax:** Validating that the code is not just grammatically correct but actually performs the task requested by the user through ground-truth validation.

4. HIGH-LEVEL SYSTEM ARCHITECTURE

4.1 Overview of the Proposed System

The proposed system is an autonomous multi-agent framework designed to generate and test Node.js REST APIs from natural language requirements. At its core—and the primary focus of this research—is the **Adaptive Orchestration Engine**.

Acting as the system's central nervous system, this engine uses a LangGraph Finite State Machine (FSM) to deterministically govern the planning, coding, and testing agents. Crucially, it houses a specialized **Diagnostic Critic Agent** paired with an **Episodic Memory** vector database. When a runtime error occurs, this engine autonomously retrieves past successful fixes, diagnoses the root cause, and synthesizes a targeted self-correction strategy. By integrating Docker sandboxing, strict "retry budgets," and Human-in-the-Loop (HITL) checkpoints, the system transforms fragile AI code generation into a secure, scalable, and self-healing software factory.

System Architecture Diagram

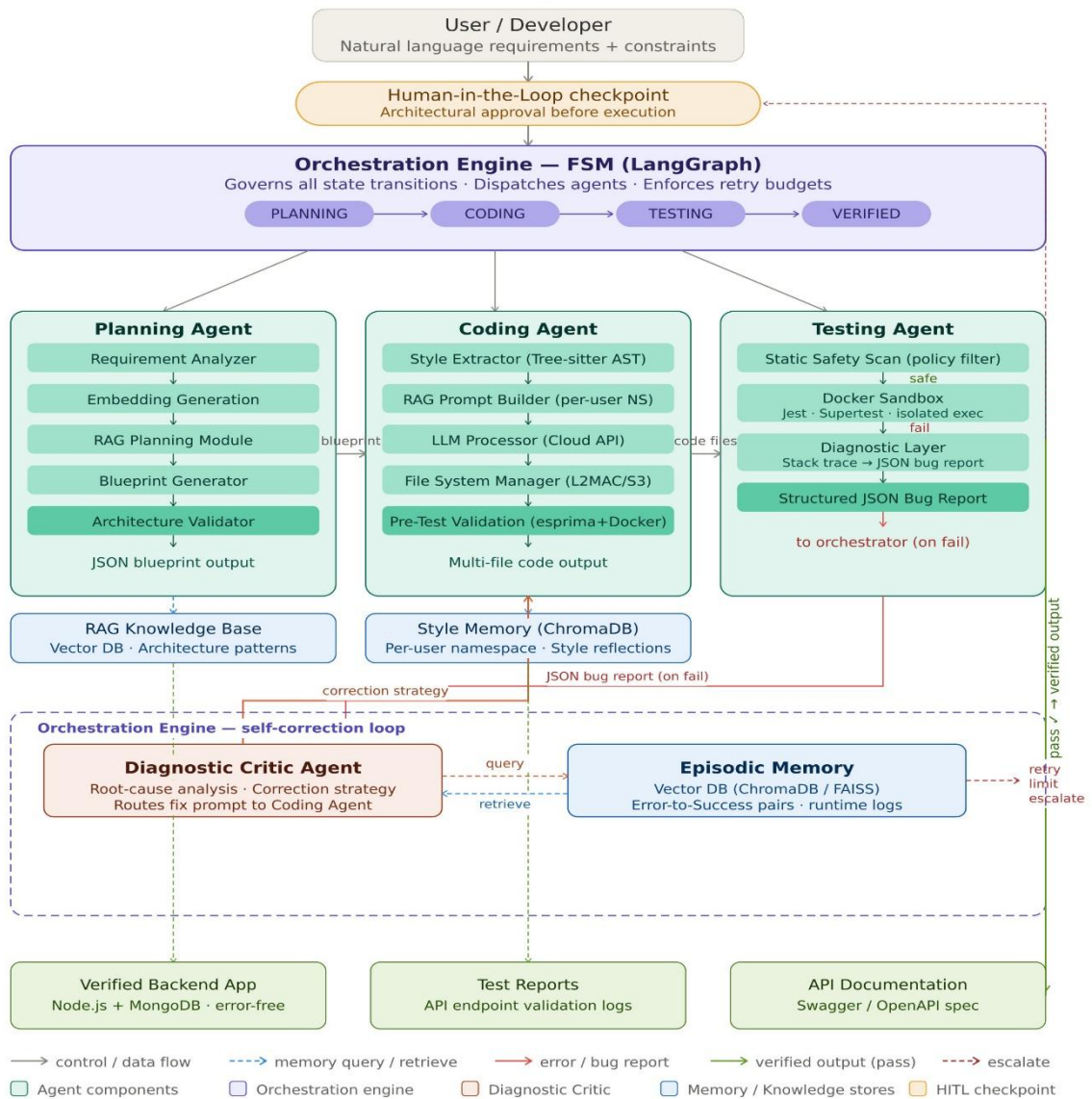


Figure 1 - System architecture Diagram

Individual Component

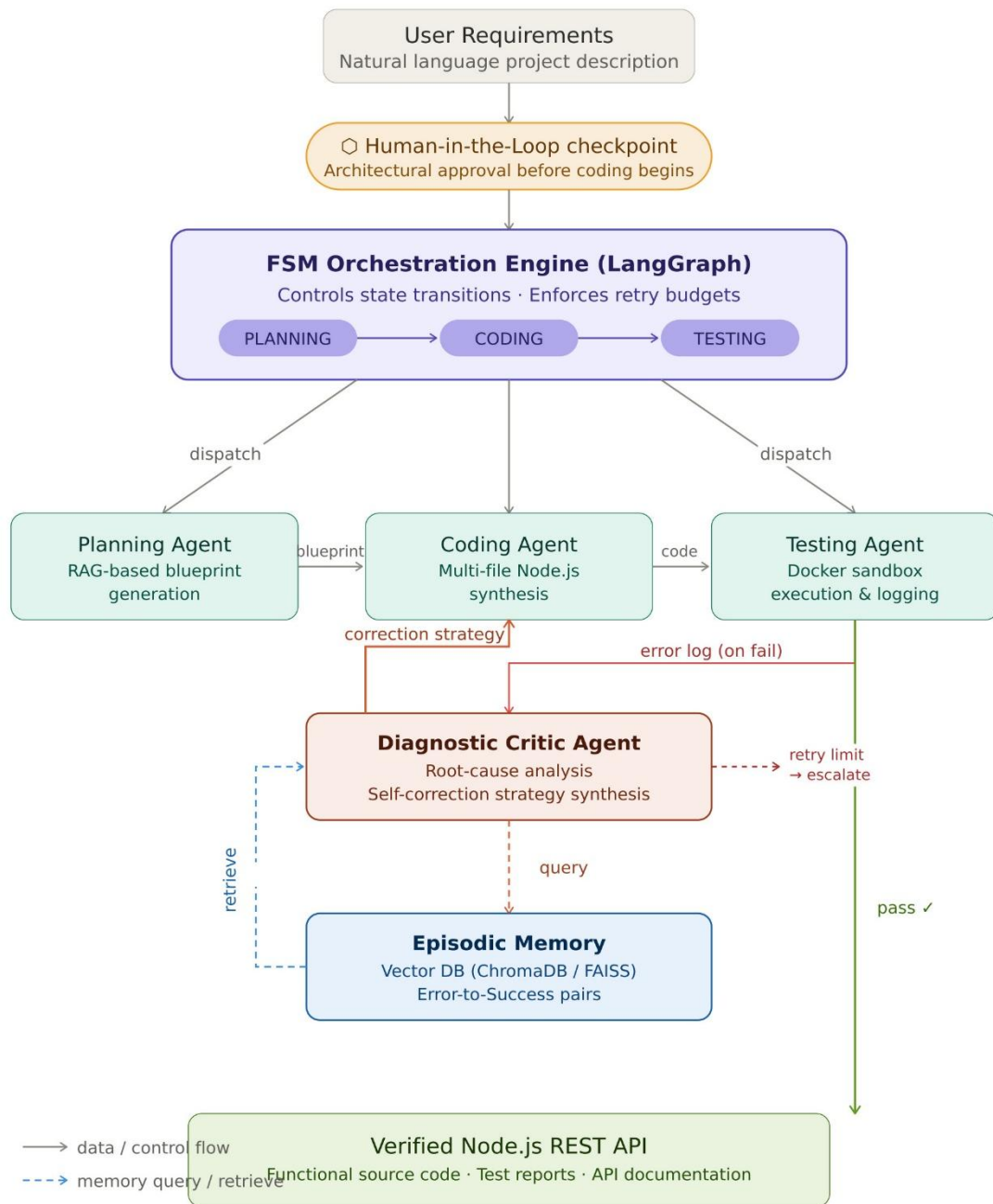


Figure 2- Individual Component

5. USER REQUIREMENTS

5.1 Stakeholder Identification

- **External User (Lead Developer):** Provides high-level project requirements and monitors system execution.
- **Internal Agents (Functional Stakeholders):** Planning Agent: Receives structural blueprints and triggers from the Orchestrator. Coding Agent: Receives refined correction strategies and retry commands. Testing Agent: Provides execution logs and error feedback to the Orchestrator

5.2 Functional Requirements

The functional requirements of the system describe the core capabilities it must provide:

1. **State Governance:** The system shall use an FSM-based workflow (LangGraph) to manage transitions between Planning, Coding, and Testing phases.
2. **Episodic Memory Capture:** The system shall store "Error-to-Success" pairs in a runtime vector database to facilitate real-time learning.
3. **Pattern Retrieval:** The system shall semantically retrieve past fix strategies based on current error patterns to prevent repetitive failures.
4. **Autonomous Repair:** The system shall generate diagnostic prompts to trigger recursive self-correction loops for the coding agent.
5. **Resource Management:** The system shall track global project status and enforce "Retry Budgets" to prevent infinite reasoning loops

5.3 Non-Functional Requirements

Performance: The orchestration engine must retrieve relevant memory matches in under 2 seconds to ensure efficient processing.

Security: All autonomous code execution and repair attempts must be restricted to an isolated Docker sandbox.

Reliability: The system must provide a "Safe Exit" with a detailed error log if the maximum retry limit is reached.

Usability: The engine must provide real-time status updates (e.g., "Verified", "Fixing", "Pending") for all software modules.

Scalability: The episodic memory system must handle a minimum of 100 error logs per session without performance degradation.

6. COMMERCIALIZATION PLAN

6.1 Target Market and Customer Persona

Primary market: Sri Lankan software development companies (SMEs and freelancing platforms) and individual backend developers maintaining multiple concurrent client projects.

Customer persona: Ravi, a 28-year-old full-stack developer at a Colombo-based startup, maintains 5–7 concurrent client projects, spends 30–40% of his time on repetitive backend code and style refactoring, and wants an API-accessible tool that learns and enforces his personal coding conventions without requiring him to manage any local AI infrastructure.

6.2 Value Proposition

Generates production-ready, style-consistent Node.js backends via a simple API call in minutes rather than hours. The only tool that automatically learns and persistently enforces a developer's personal coding conventions commercial tools do not do this. API-accessible and integrable into any IDE, CLI, or CI/CD pipeline with no local setup required. Open-source core with a transparent, auditable architecture unlike the black-box commercial tools. Multi-tenant architecture means teams can share one deployment while each developer retains their own fully isolated style profile.

6.3 Revenue Model

Open-source core (MIT licence, GitHub) free for students, individuals, and small teams; builds community adoption and trust. Premium Enterprise Edition (paid) custom style profiles, team shared style libraries, VS Code extension with one-click integration, priority support, and SLA guarantees (LKR 2,500–5,000 per month per team). Freemium API model limited monthly generation requests free; higher request quotas, multi-project episodic memory, and auto-Swagger with Docker

6.4 Cost Estimation

One-time individual licence: LKR 15,000. Monthly developer subscription: LKR 1,500. Team bundle (up to 10 developers): LKR 10,000 per month. Training workshops for IT companies: LKR 25,000 per session.

6.5 Competitive Advantage

Unlike Cursor, Replit Agent, and Claude Code — which are style-agnostic and treat all developers identically — this agent learns each developer's personal conventions once from their own repository and enforces them permanently across all future projects via a persistent namespace RAG profile. The multi-tenant architecture means this personalization scales to many users on shared infrastructure with guaranteed isolation. This combination is uniquely absent from all commercial and academic tools in the current landscape.

7. BUDGET AND JUSTIFICATION

The estimated budget for the proposed research project includes costs related to hardware resources, cloud computing services, and professional human effort. These resources are essential to support the complex orchestration of LLMs, model fine-tuning, and the evaluation of the episodic memory system

7.1 Hardware Costs

- High-Speed External SSD for Model & Data Backup: 15,000
- Networking & High-Speed Interface Cables: 5,000
- **Subtotal: 20,000**

Justification: High-speed external storage is critical for securely archiving local LLM weights, extensive execution logs, and the Vector Database embeddings that power the Episodic Memory. Reliable backup infrastructure ensures project continuity, while networking peripherals facilitate stable, high-bandwidth data transfers between local development environments and cloud GPU instances.

7.2 Software License Costs

- Premium Development Plugins and API Tools: 5,000
- Data Visualization and Reporting Tools: 5,000
- **Subtotal: 10,000**

Justification: While the core multi-agent architecture utilizes open-source frameworks, minimal capital is allocated for specialized IDE plugins and advanced visualization tools. These are essential for monitoring real-time state transitions, evaluating the LangGraph workflow, and debugging the complex recursive logic within the Diagnostic Critic Agent.

7.3 Cloud Service Costs

- GPU Cloud Computing for Model Fine-tuning: 45,000
- Proprietary API Credits (GPT-4o / Claude 3.5): 30,000
- **Subtotal: 75,000**

Justification: Fine-tuning localized models (e.g., DeepSeek) for specialized Node.js REST API synthesis requires substantial VRAM that exceeds standard local hardware capabilities, necessitating cloud-based GPU provisioning. Furthermore, API credits for frontier proprietary models are required to establish robust performance baselines, allowing for a comparative evaluation of the Critic Agent's autonomous self-correction accuracy.

7.4 Human Effort Cost

- Algorithm Engineering & System Orchestration: 80,000
- Technical Implementation & System Validation: 70,000
- **Subtotal: 150,000**

Justification: The human effort allocation reflects the intensive technical expertise and architectural design invested by the research team. This encompasses engineering the LangGraph Finite State Machine, constructing the active Episodic Memory vector store, and developing the specialized Diagnostic Critic Agent. While conducted in an academic setting, this notional cost quantifies the real-world professional value of the hours required to achieve full system autonomy.

7.5 Total Estimated Budget

Resource Category	Estimated Cost
Hardware Costs	20,000
Software License Costs	10,000
Cloud Service Costs	75,000
Human Effort Cost	150,000
Total Estimated Budget	255,000

Table 2 - Total Estimated Budget

8. WORK BREAKDOWN STRUCTURE (WBS)

8.1 Hierarchical Task Breakdown

Member 1: Context-Aware Planning & RAG Architecture

- Establishment of the local Vector Database (Chroma/FAISS) for architectural pattern storage.
- Development of semantic retrieval logic for few-shot prompt enhancement.
- Implementation of the JSON technical blueprint generation engine.
- Validation of architectural schemas to ensure strict compliance with Node.js REST API standards.

Member 2: Generative Coding & Module Synthesis

- Development of the advanced prompt engineering pipeline for multi-file code generation.
- Implementation of the File System Manager for automated directory and file creation.
- Injection of business logic and dependency management into generated modules.
- Integration of automated syntax validation algorithms prior to execution.

Member 3: Autonomous QA & Sandboxed Execution

- Configuration of the isolated, Docker-based execution environment.
- Development of the automated Unit and Integration test runners.
- Implementation of runtime log capture and detailed execution trace generation.
- Creation of a safety analysis layer for detecting non-terminating code patterns and infinite loops.

Member 4: Adaptive Orchestration & Episodic Memory (Individual Focus)

- Design of the deterministic workflow graph using LangGraph to govern multi-agent state transitions.
- Construction of the active Runtime Vector Store to index episodic memory logs ("Error-to-Success" pairs).
- Development of the **Diagnostic Critic Agent** for advanced self-correction prompting and root-cause bug diagnosis.
- Implementation of global state persistence and strict retry budget management to prevent agentic mode collapse.

8.2 WBS Visual Representation (Diagram Blueprint)

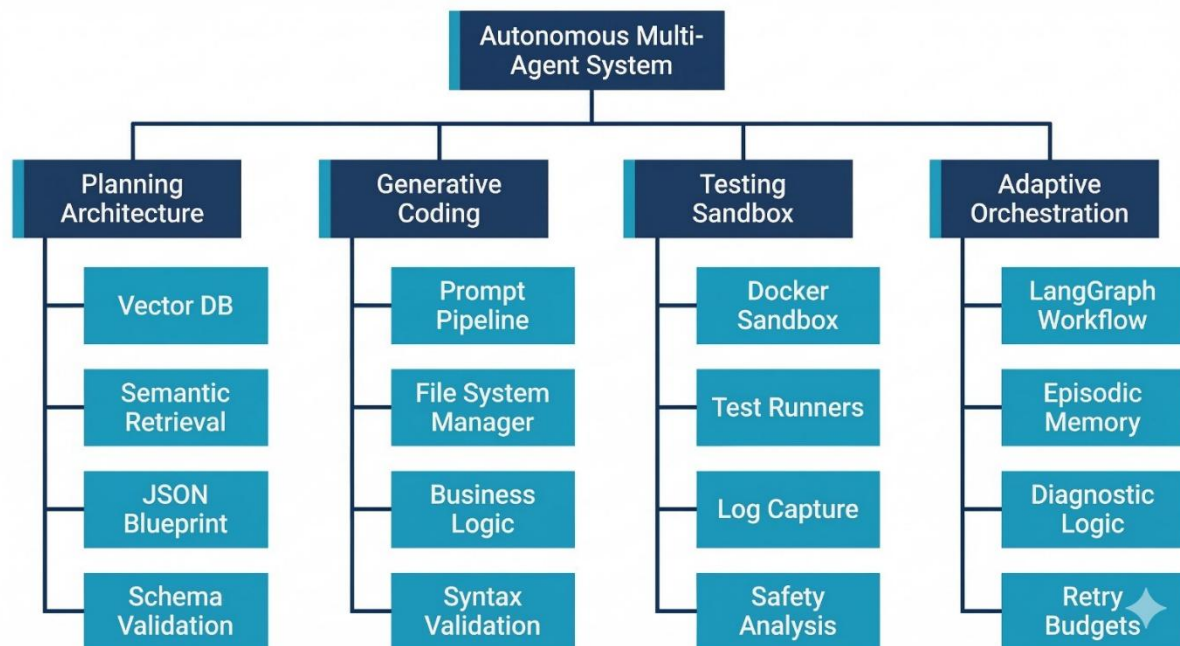


Figure 3- Work Breakdown Structure

8.3 Timeline Alignment

Phase 1: Research and Planning Focuses on comparing different AI models to see which is best for Node.js. This phase ends with the formal project proposal submission.

Phase 2: Building the Parts Each member builds their own component (Planning, Coding, Testing, and Orchestration) separately to make sure they work on a basic level.

Phase 3: System Integration The individual parts are connected together to create a single workflow where a plan is turned into code and then tested automatically.

Phase 4: Brain and Memory Development The focus shifts to the "Episodic Memory," giving the system the ability to remember its mistakes and try to fix them without human help.

Phase 5: Testing and Final Report The complete system is tested to see how well it works. All data is collected, and the final research report is written and submitted.

Phase 6: Final Presentation The project ends with a final presentation (Viva) to the university and the submission of a research paper based on the results.

9. GANTT CHART

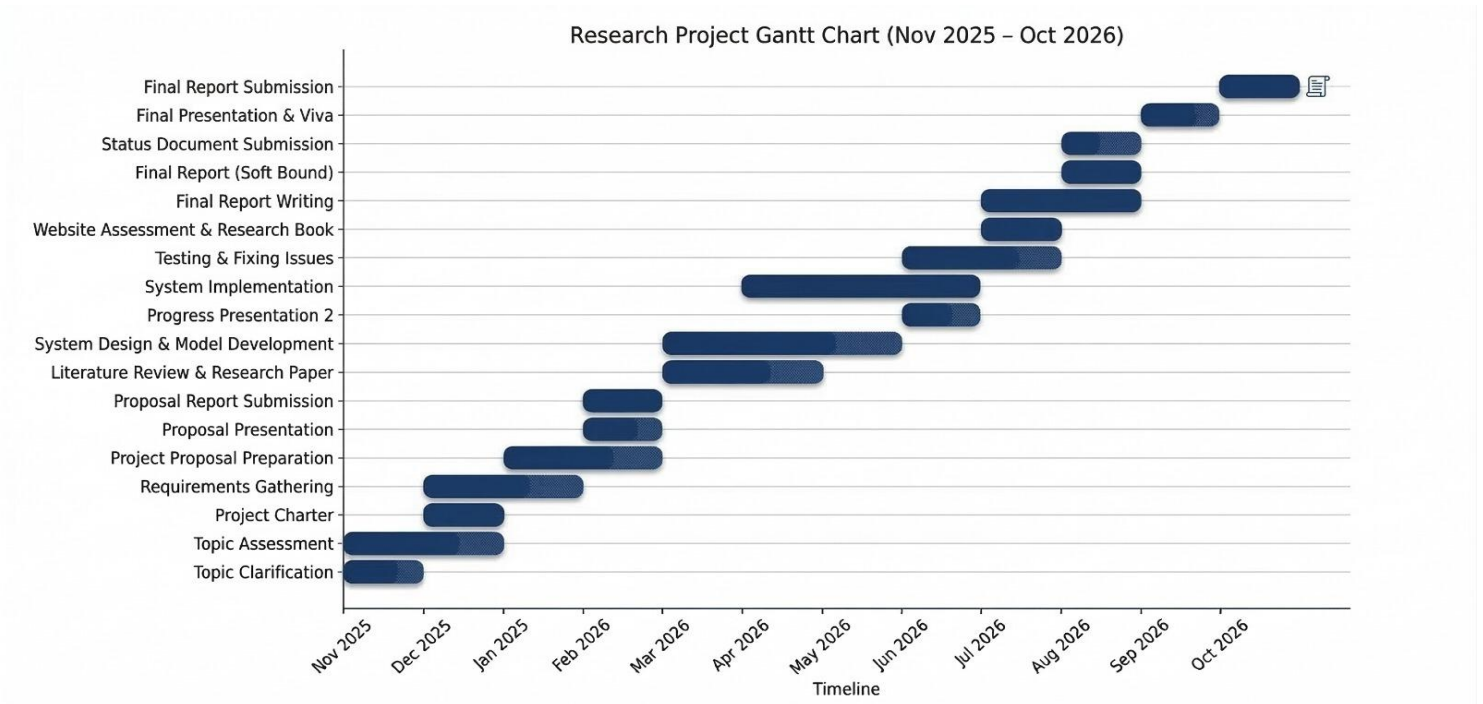


Figure 4 - GANTT Chart

10. REFERENCES LIST

[1] Multi-Agent Frameworks & Orchestration

- [1] C. Qian *et al.*, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meet. Assoc. Comput. Linguist. (ACL)*, 2024, pp. 15174–15186, doi: 10.18653/v1/2024.acl-long.810.
- [2] S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024
- [3] D. Huang, Q. Bu, J. Zhang, X. Xie, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," *arXiv preprint arXiv:2312.13010*, 2024.
- [4] L. Lin, Y. Jin, Y. Zhou, W. Chen, and C. Qian, "MAO: A framework for process model generation with multi-agent orchestration," *IEEE Trans. Serv. Comput.*, vol. 18, no. 6, pp. 3987–3999, Nov./Dec. 2025, doi: 10.1109/TSC.2025.3611816.
- [11] Y. Zhang, X. Liu, and C. Xiao, "MetaAgent: Automatically constructing multi-agent systems based on finite state machines," *arXiv preprint arXiv:2507.22606*, 2025.

[2] Self-Correction & Reasoning

- [5] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.
- [6] T. X. Olausson, J. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [13] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[3] State Machines & Workflows

- [7] Y. Wu, S. Zhang, Q. Wu, T. Yue, and C. Wang, "StateFlow: Enhancing LLM task-solving through state-driven workflows," in *Proc. Conf. Lang. Modeling (COLM)*, 2024.
- [10] R. Qi *et al.*, "Memory-augmented state machine prompting: A novel LLM agent framework for real-time strategy games," *arXiv preprint arXiv:2510.18395*, 2025.

11.APPENDICES

APPENDIX A: Risk Assessment and Mitigation

The following matrix identifies the potential technical risks associated with the Adaptive Orchestration Engine and outlines the corresponding mitigation strategies.

- **Risk: Infinite Execution Loops (Agentic Mode Collapse)**
 - **Mitigation:** Implementation of a strict "Retry Budget" governed by the LangGraph Finite State Machine (FSM). If the Diagnostic Critic Agent exhausts its allocated attempts to resolve a bug, the FSM actively halts the automated loop and escalates the unresolved error to the human user.
- **Risk: Memory Hallucination & Suboptimal Self-Correction**
 - **Description:** The system's episodic memory may retrieve a historical fix that is contextually irrelevant to the current codebase, potentially introducing cascading errors.
 - **Mitigation:** Every repair strategy synthesized by the Critic Agent is treated as a hypothesis. It must successfully pass the automated testing sandbox before the FSM permanently updates the global project state or commits the new code to the file system.
- **Risk: System Resource Exhaustion (Context Window Limits)**
 - **Mitigation:** Utilization of ranked semantic retrieval via the Vector Database to strictly limit the LLM context window usage. Additionally, the architecture supports localized model execution to manage API costs and computational overhead.
- **Risk: Security Vulnerabilities in Generated Code**
 - **Mitigation:** Enforcement of strict hardware isolation using Docker containerization. This guarantees that any malicious, hallucinated, or non-terminating system commands generated by the coding agent cannot escape the sandbox to affect the host machine.

APPENDIX B: Ethical Compliance and Data Governance

This section outlines the ethical considerations and strict data handling protocols governing the research.

- **Open-Source Integrity:** All reference datasets and architectural templates utilized by the RAG pipeline are strictly sourced from repositories carrying permissive, open-source licenses (e.g., MIT or Apache 2.0).
- **Human Agency (HITL):** A mandatory "Human-in-the-Loop" checkpoint is hardcoded into the FSM. This ensures that the AI cannot unilaterally finalize high-level architectural blueprints without explicit human review and approval, preserving developer agency.

- **Data Privacy & Sanitization:** The orchestration engine actively filters runtime execution logs to guarantee that no sensitive user data, personally identifiable information (PII), or environment secrets (e.g., API keys, database passwords) are persistently indexed within the Episodic Memory vector store.
- **Intellectual Property:** All Node.js source code synthesized by the multi-agent framework is classified as the intellectual property of the human developer or end-user, with the AI operating strictly as an autonomous generative utility.